

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



ANNISA BERLIANA NINDYA SYAH PUTRI

109082500166

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Pengurutan Data (Sorting)

Sorting merupakan proses mengatur sekumpulan data agar tersusun rapi sesuai kriteria tertentu. Biasanya pengurutan dilakukan berdasarkan nilai kunci (key) dari setiap elemen, sehingga memudahkan proses pencarian maupun analisis data selanjutnya.

Secara umum, urutan data dapat dibedakan menjadi dua jenis:

- **Ascending (menaik)** → data disusun dari nilai terkecil ke terbesar
- **Descending (menurun)** → data disusun dari nilai terbesar ke terkecil

Dalam dunia algoritma, terdapat banyak metode sorting yang bisa digunakan. Dua metode sederhana yang sering dipelajari adalah *Selection Sort* dan *Insertion Sort*. Kedua algoritma ini memiliki kompleksitas waktu sebesar $O(n^2)$ pada kondisi rata-rata maupun terburuk, sehingga kurang efisien untuk data berukuran besar, tetapi tetap penting untuk memahami dasar logika pengurutan.

2. Selection Sort

Selection Sort adalah metode pengurutan yang bekerja dengan cara memilih elemen tertentu dari bagian data yang belum terurut, lalu menempatkannya ke posisi yang benar.

Prinsip utamanya adalah:

- Menentukan nilai terkecil (untuk ascending) atau terbesar (untuk descending) dari data yang belum terurut
- Menukar posisi nilai tersebut dengan elemen di bagian depan

Proses ini dilakukan secara berulang hingga seluruh data tersusun dengan benar.

Alur kerja Selection Sort (ascending):

- a) Tentukan nilai minimum dari seluruh elemen yang belum terurut
- b) Tukar nilai tersebut dengan elemen di posisi awal bagian yang belum terurut
- c) Geser batas awal ke kanan dan ulangi langkah yang sama
- d) Proses berhenti ketika seluruh elemen sudah berada di posisi yang tepat

Metode ini sederhana karena hanya fokus pada satu elemen terbaik di setiap iterasi, tetapi jumlah perbandingannya cukup banyak.

3. Insertion Sort

Insertion Sort adalah algoritma yang mengurutkan data dengan cara menyisipkan elemen ke dalam posisi yang sesuai pada bagian data yang sudah terurut.

Berbeda dengan Selection Sort, metode ini tidak mencari nilai ekstrem dari seluruh data. Sebaliknya, algoritma ini mengambil satu elemen, lalu membandingkannya dengan elemen sebelumnya hingga menemukan posisi yang tepat.

Selama proses ini, elemen-elemen yang lebih kecil atau lebih besar akan digeser untuk memberi ruang bagi elemen yang sedang diproses.

Langkah kerja Insertion Sort (descending):

- a. Anggap elemen pertama sudah dalam kondisi terurut
- b. Ambil elemen berikutnya sebagai data yang akan diproses
- c. Bandingkan dengan elemen di sebelah kiri
- d. Geser elemen yang tidak sesuai posisinya
- e. Sisipkan elemen pada posisi yang tepat
- f. Ulangi hingga semua data terurut

Insertion Sort cenderung lebih efisien dibanding Selection Sort untuk data yang sudah hampir terurut karena jumlah pergeseran yang lebih sedikit.

B. Guided

1. Program Pengurutan Data Menggunakan Selection Sort

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int
    for i = 0; i < len(arr)-1; i++ { //perulangan luar
        idx_min = i
        for j = i + 1; j < len(arr); j++ { //perulangan dalam
            if arr[j] < arr[idx_min] { //kondisional
                idx_min = j
            }
        }
    }
}
```

```

    }
}
    if idx_min != i { //swap
        arr[i], arr[idx_min] = arr[idx_min], arr[i]
    }
}
}
func main() {
    var arrAngka [8]int
    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d: ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()
    SelectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Guided\Guided-1\guided1.go"
Masukkan data angka indeks ke-0: 1
Masukkan data angka indeks ke-1: 3
Masukkan data angka indeks ke-2: 4
Masukkan data angka indeks ke-3: 5
Masukkan data angka indeks ke-4: 6
Masukkan data angka indeks ke-5: 2
Masukkan data angka indeks ke-6: 4
Masukkan data angka indeks ke-7: 6

=== SEBELUM SORTING ===
1 3 4 5 6 2 4 6
=== SETELAH SORTING ===
1 2 3 4 4 5 6 6

```

2. Program Pengurutan Data Mahasiswa Berdasarkan IPK Menggunakan Selection Sort

```
package main
```

```

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int

    for i = 0; i < len(sMHS)-1; i++ { //loop Luar
        idx_min = i

        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari
            nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil
                ke terbesar
                idx_min = j
            }
        }

        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n", i)
        fmt.Print("Nama: ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM: ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK: ")
        fmt.Scan(&structMHS[i].IPK)
    }

    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
    }
}

```

```

        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }

    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)

    fmt.Println("=== SETELAH SORTING ===")

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }

    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Guided\Guided-1\Guided-2\guided2.go"
--- Masukkan Data Mahasiswa ke-0 ---
Nama: mawar
NIM: 143
IPK: 3.88
--- Masukkan Data Mahasiswa ke-1 ---
Nama: melati
NIM: 675
IPK: 3.46
--- Masukkan Data Mahasiswa ke-2 ---
Nama: anggrek
NIM: 897
IPK: 3.65
--- Masukkan Data Mahasiswa ke-3 ---
Nama: tulip
NIM: 109
IPK: 3.77
--- Masukkan Data Mahasiswa ke-4 ---
Nama: jeruk
NIM: 327
IPK: 3.00

```

```
=== SEBELUM SORTING ===  
--- Data Mahasiswa ke-0 ---  
Nama: mawar  
NIM: 143  
IPK: 3.88  
--- Data Mahasiswa ke-1 ---  
Nama: melati  
NIM: 675  
IPK: 3.46  
--- Data Mahasiswa ke-2 ---  
Nama: anggrek  
NIM: 897  
IPK: 3.65  
--- Data Mahasiswa ke-3 ---  
Nama: tulip  
NIM: 109  
IPK: 3.77  
--- Data Mahasiswa ke-4 ---  
Nama: jeruk  
NIM: 327  
IPK: 3.00  
=====
```

```
=== SETELAH SORTING ===  
--- Data Mahasiswa ke-0 ---  
Nama: jeruk  
NIM: 327  
IPK: 3.00  
--- Data Mahasiswa ke-1 ---  
Nama: melati  
NIM: 675  
IPK: 3.46  
--- Data Mahasiswa ke-2 ---  
Nama: anggrek  
NIM: 897  
IPK: 3.65  
--- Data Mahasiswa ke-3 ---  
Nama: tulip  
NIM: 109  
IPK: 3.77  
--- Data Mahasiswa ke-4 ---  
Nama: mawar  
NIM: 143  
IPK: 3.88  
=====
```

3. Program Pengurutan Data Mahasiswa Berdasarkan IPK Menggunakan Selection Sort

```
package main  
  
import "fmt"  
  
type arrInt [100]int  
  
func insertionSort(T *arrInt, n int) {  
    var temp, i, j int
```

```

    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j--
        }

        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data: ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("\nData sebelum sorting: ")

    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting (Descending): ")

    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}

```


Output :

```
PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Guided\Guided-3\guided3.go"
Masukkan jumlah data: 3
Masukkan data:
21 8 9

Data sebelum sorting:
21 8 9

Data setelah sorting (Descending):
21 9 8
```

4. Program Pengurutan Data Mahasiswa Berdasarkan Nama Menggunakan Insertion Sort (Descending)

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   string
    ipk   float64
}

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1

        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j--
        }

        T[j+1] = temp
    }
}

func main() {
    var data arrMhs
```

```
var n, i int

fmt.Print("Masukkan jumlah mahasiswa: ")
fmt.Scan(&n)

for i = 0; i < n; i++ {
    fmt.Println("\nData mahasiswa ke-", i+1)
    fmt.Print("Nama: ")
    fmt.Scan(&data[i].nama)
    fmt.Print("NIM: ")
    fmt.Scan(&data[i].nim)
    fmt.Print("IPK: ")
    fmt.Scan(&data[i].ipk)
}

fmt.Println("\nData sebelum sorting: ")

for i = 0; i < n; i++ {
    fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
}

insertionSortStruct(&data, n)

fmt.Println("\nData setelah sorting (Descending berdasarkan
Nama): ")

for i = 0; i < n; i++ {
    fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
}
}
```

Output :

```
PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Guided\Guided-4\guided4.go"
Masukkan jumlah mahasiswa: 3

Data mahasiswa ke- 1
Nama: mawar
NIM: 234
IPK: 3.25

Data mahasiswa ke- 2
Nama: melati
NIM: 123
IPK: 3.89

Data mahasiswa ke- 3
Nama: anggrek
NIM: 567
IPK: 2.78

Data sebelum sorting:
mawar 234 3.25
melati 123 3.89
anggrek 567 2.78

Data setelah sorting (Descending berdasarkan Nama):
melati 123 3.89
mawar 234 3.25
anggrek 567 2.78
```

C. Unguided

1. Soal Unguided 1

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(data []int, jumlah int) {
    for i := 0; i < jumlah-1; i++ {
        posMin := i

        for j := i + 1; j < jumlah; j++ {
            if data[j] < data[posMin] {
                posMin = j
            }
        }

        temp := data[posMin]
        data[posMin] = data[i]
        data[i] = temp
    }
}

func main() {
    var banyakKasus int
    fmt.Scan(&banyakKasus)

    for i := 0; i < banyakKasus; i++ {
        var jumlahData int
        fmt.Scan(&jumlahData)

        var angka []int

        for j := 0; j < jumlahData; j++ {
            var inputAngka int
            fmt.Scan(&inputAngka)
            angka = append(angka, inputAngka)
        }

        selectionSort(angka, jumlahData)

        for j := 0; j < jumlahData; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
        }
    }
}
```

```

        fmt.Print(angka[j])
    }

    fmt.Println()
}
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Unguided\unguided1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Program ini dibuat untuk mengurutkan beberapa data angka menggunakan metode Selection Sort. Di awal, program membaca berapa banyak kasus yang akan dijalankan. Pada setiap kasus, pengguna diminta memasukkan jumlah data, lalu menginput angka satu per satu yang kemudian disimpan dalam sebuah array dinamis (slice).

Setelah data terkumpul, dilakukan proses pengurutan. Caranya yaitu dengan mencari nilai terkecil dari bagian data yang belum terurut, lalu menukarnya dengan elemen di posisi awal bagian tersebut. Langkah ini diulang terus sampai seluruh data tersusun rapi dari nilai terkecil ke terbesar. Terakhir, hasil pengurutan ditampilkan untuk setiap kasus yang telah diproses.

2. Soal Unguided 2

- 2) Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main

import "fmt"

func sortNaik(data []int, panjang int) {
    for i := 0; i < panjang-1; i++ {
        posMin := i
        for j := i + 1; j < panjang; j++ {
            if data[j] < data[posMin] {
                posMin = j
            }
        }
        data[i], data[posMin] = data[posMin], data[i]
    }
}
```

```

    }
    }
    temp := data[posMin]
    data[posMin] = data[i]
    data[i] = temp
}
}

func sortTurun(data []int, panjang int) {
    for i := 0; i < panjang-1; i++ {
        posMax := i
        for j := i + 1; j < panjang; j++ {
            if data[j] > data[posMax] {
                posMax = j
            }
        }
        temp := data[posMax]
        data[posMax] = data[i]
        data[i] = temp
    }
}

func main() {
    var jumlahKasus int
    fmt.Scan(&jumlahKasus)

    for i := 0; i < jumlahKasus; i++ {
        var banyakData int
        fmt.Scan(&banyakData)

        var dataGanjil []int
        var dataGenap []int

        for j := 0; j < banyakData; j++ {
            var angkaInput int
            fmt.Scan(&angkaInput)

            if angkaInput%2 != 0 {
                dataGanjil = append(dataGanjil, angkaInput)
            } else {
                dataGenap = append(dataGenap, angkaInput)
            }
        }

        sortNaik(dataGanjil, len(dataGanjil))
    }
}

```

```

        sortTurun(dataGenap, len(dataGenap))

        awal := true

        for j := 0; j < len(dataGanjil); j++ {
            if !awal {
                fmt.Print(" ")
            }
            fmt.Print(dataGanjil[j])
            awal = false
        }

        for j := 0; j < len(dataGenap); j++ {
            if !awal {
                fmt.Print(" ")
            }
            fmt.Print(dataGenap[j])
            awal = false
        }

        fmt.Println()
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Unguided\Unguided-2\unguided2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Penjelasan :

Program ini digunakan untuk mengelompokkan dan mengurutkan bilangan ganjil dan genap. Data yang dimasukkan akan dipisahkan terlebih dahulu menjadi dua bagian, yaitu bilangan ganjil dan bilangan genap. Setelah itu, bilangan ganjil diurutkan secara menaik, sedangkan bilangan genap diurutkan secara menurun menggunakan metode Selection Sort. Hasil akhirnya ditampilkan dengan urutan bilangan ganjil terlebih dahulu, kemudian diikuti bilangan genap dalam satu baris output.

3. Soal Unguided 3

- 3) Kompetisi pemrograman yang baru saja berlalu diikuti oleh 17 tim dari berbagai perguruan tinggi ternama. Dalam kompetisi tersebut, setiap tim berlomba untuk menyelesaikan sebanyak mungkin problem yang diberikan. Dari 13 problem yang diberikan, ada satu problem yang menarik. Problem tersebut mudah dipahami, hampir semua tim mencoba untuk menyelesaikannya, tetapi hanya 3 tim yang berhasil. Apa sih problemnya?

"Median adalah nilai tengah dari suatu koleksi data yang sudah terurut. Jika jumlah data genap, maka nilai median adalah rerata dari kedua nilai tengahnya. Pada problem ini, semua data merupakan bilangan bulat positif, dan karenanya rerata nilai tengah dibulatkan ke bawah."

Buatlah program **median** yang mencetak nilai median terhadap seluruh data yang sudah terbaca, jika data yang dibaca saat itu adalah 0.

Masukan berbentuk rangkaian bilangan bulat. Masukan tidak akan berisi lebih dari 1000000 data, tidak termasuk bilangan 0. Data 0 merupakan tanda bahwa median harus dicetak, tidak termasuk data yang dicari mediannya. Data masukan diakhiri dengan bilangan bulat -5313.

Keluaran adalah median yang diminta, satu data per baris.

No	Masukan	Keluaran
1	7 23 11 0 5 19 2 29 3 13 17 0 -5313	11 12

Keterangan:

Sampai bilangan 0 yang pertama, data terbaca adalah 7 23 11, setelah tersusun: 7 11 23, maka median saat itu adalah 11.

Sampai bilangan 0 yang kedua, data adalah 7 23 11 5 19 2 29 3 13 17, setelah tersusun diperoleh: 2 3 5 7 **11 13** 17 19 23 29. Karena ada 10 data, genap, maka median adalah $(11+13)/2=12$.

Petunjuk:

Untuk setiap data bukan 0 (dan bukan marker -5313541) simpan ke dalam array, Dan setiap kali menemukan bilangan 0, urutkanlah data yang sudah tersimpan dengan menggunakan metode insertion sort dan ambil mediannya.

Jawaban :

```

package main

import "fmt"

func insertionSort(arr []int, n int) {
    var i, j, temp int

    for i = 1; i < n; i++ {
        temp = arr[i]
        j = i - 1

        for j >= 0 && arr[j] > temp {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = temp
    }
}

func hitungMedian(arr []int, n int) int {
    if n%2 == 1 {
        return arr[n/2]
    } else {
        return (arr[n/2-1] + arr[n/2]) / 2
    }
}

func main() {
    var data []int

    for {
        var input int
        fmt.Scan(&input)

        if input == -5313 {
            break
        }

        if input == 0 {
            insertionSort(data, len(data))
            median := hitungMedian(data, len(data))
            fmt.Println(median)
        } else {
            data = append(data, input)
        }
    }
}

```

```

    }
}
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Unguided\Unguided-3\unguided3.go"
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12

```

Penjelasan :

Program ini membaca data angka satu per satu hingga menemukan tanda akhir. Setiap angka yang bukan 0 akan disimpan ke dalam array. Ketika program membaca angka 0, seluruh data yang sudah tersimpan akan diurutkan menggunakan metode insertion sort. Setelah itu, median dihitung berdasarkan jumlah data yang ada, yaitu mengambil nilai tengah jika jumlah data ganjil, atau rata-rata dua nilai tengah (dibulatkan ke bawah) jika jumlah data genap. Proses ini dilakukan setiap kali angka 0 muncul hingga program berhenti saat membaca nilai -5313.

4. Soal Unguided 4

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	32 13 25 49 1 7 19 37 5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main

import "fmt"

func urutInsertion(data []int, panjang int) {
    for i := 1; i < panjang; i++ {
        simpan := data[i]
        pos := i

        for pos > 0 && simpan < data[pos-1] {
            data[pos] = data[pos-1]
            pos = pos - 1
        }

        data[pos] = simpan
    }
}

func main() {
    var angka []int
    var nilai int

    for {
        fmt.Scan(&nilai)

        if nilai < 0 {
            break
        }

        angka = append(angka, nilai)
    }

    jumlah := len(angka)

    if jumlah > 0 {
        urutInsertion(angka, jumlah)
    }

    for i := 0; i < jumlah; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(angka[i])
    }
}
```

```

    }
    fmt.Println()

    if jumlah < 2 {
        fmt.Println("Data berjarak tidak tetap")
    } else {
        selisih := angka[1] - angka[0]
        konsisten := true

        for i := 1; i < jumlah-1; i++ {
            if angka[i+1]-angka[i] != selisih {
                konsisten = false
                break
            }
        }

        if konsisten {
            fmt.Printf("Data berjarak %d\n", selisih)
        } else {
            fmt.Println("Data berjarak tidak tetap")
        }
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Unguided\Unguided-4\unguided4.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
● go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Unguided\Unguided-4\unguided4.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan :

Program ini digunakan untuk membaca sejumlah data bilangan hingga ditemukan nilai negatif sebagai tanda berhenti. Data yang telah dimasukkan kemudian diurutkan menggunakan metode insertion sort dari nilai terkecil ke terbesar. Setelah proses pengurutan selesai, program menampilkan data tersebut dalam satu baris. Selanjutnya, program mengecek apakah selisih antar data berurutan bernilai sama. Jika selisihnya konsisten, maka akan ditampilkan

besar selisih tersebut. Jika tidak, program akan menyatakan bahwa data memiliki jarak yang tidak tetap.

5. Soal Unguided 5

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```
procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n : integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
 F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n : integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
 F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
 terfavorit, yaitu memiliki rating tertinggi}

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer )
{I.S. Array pustaka berisi n data buku
 F.S. Array pustaka terurut menurun/mengecil terhadap rating.
 Catatan: Gunakan metoda Insertion sort}

procedure Cetak5Terbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan 5 judul buku dengan rating tertinggi
 Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r : integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
 eksemplar, rating) dengan rating yang diberikan. Jika tidak ada buku
 dengan rating yang ditanyakan, cukup tuliskan "Tidak ada buku dengan
 rating seperti itu". Catatan: Gunakan pencarian biner/belah dua.}
```

Jawaban :

```

package main

import "fmt"

const maxData int = 7919

type Buku struct {
    kode, judul, penulis, penerbit string
    jumlah, tahun, nilai           int
}

type KoleksiBuku [maxData]Buku

func inputBuku(data *KoleksiBuku, total *int) {
    fmt.Scan(total)

    for i := 0; i < *total; i++ {
        fmt.Scan(&data[i].kode, &data[i].judul,
            &data[i].penulis,
            &data[i].penerbit, &data[i].jumlah,
            &data[i].tahun, &data[i].nilai)
    }
}

func tampilTerbaik(data KoleksiBuku, total int) {
    if total == 0 {
        return
    }

    maks := data[0].nilai
    idx := 0

    for i := 1; i < total; i++ {
        if data[i].nilai > maks {
            maks = data[i].nilai
            idx = i
        }
    }

    fmt.Printf("%s %s %s %d\n", data[idx].judul,
        data[idx].penulis, data[idx].penerbit, data[idx].tahun)
}

func urutData(data *KoleksiBuku, total int) {
    for i := 1; i < total; i++ {

```

```

        simpan := data[i]
        pos := i

        for pos > 0 && simpan.nilai > data[pos-1].nilai {
            data[pos] = data[pos-1]
            pos--
        }

        data[pos] = simpan
    }
}

func tampil5Teratas(data KoleksiBuku, total int) {
    batas := 5

    if total < 5 {
        batas = total
    }

    for i := 0; i < batas; i++ {
        fmt.Println(data[i].judul)
    }
}

func cariBerdasarkanRating(data KoleksiBuku, total int, target int) {
    kiri := 0
    kanan := total - 1
    ketemu := false
    posisi := -1

    for kiri <= kanan && !ketemu {
        tengah := (kiri + kanan) / 2

        if data[tengah].nilai == target {
            ketemu = true
            posisi = tengah
        } else if target > data[tengah].nilai {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    if ketemu {
        fmt.Printf("%s %s %s %d %d %d\n", data[posisi].judul,

```



```

        data[posisi].penulis, data[posisi].penerbit,
data[posisi].tahun,
        data[posisi].jumlah, data[posisi].nilai)
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

func main() {
    var dataBuku KoleksiBuku
    var jumlah, cari int

    inputBuku(&dataBuku, &jumlah)
    tampilTerbaik(dataBuku, jumlah)
    urutData(&dataBuku, jumlah)
    tampil5Teratas(dataBuku, jumlah)

    fmt.Scan(&cari)
    cariBerdasarkanRating(dataBuku, jumlah, cari)
}

```

Output :

```

PS C:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12>
go run "c:\Users\Asus\Documents\LAPRAK ALPRO 2\109082500166-Annisa-Berliana-Nindya-Syah-Putri\Week 12\Unguided\Unguided-5\unguided5.go"
5
B01 LaskarPelangi AndreaHirata Bentang 10 2005 90
B02 Bumi TereLiye Gramedia 8 2014 85
B03 Dilan PidiBaiq PastelBooks 12 2014 88
B04 AyatAyatCinta Habiburrahman Republika 7 2004 92
B05 PerahuKertas DeeLestari Bentang 9 2009 87
AyatAyatCinta Habiburrahman Republika 2004
AyatAyatCinta
LaskarPelangi
Dilan
PerahuKertas
Bumi
88
Dilan PidiBaiq PastelBooks 2014 12 88

```

Penjelasan :

Program ini digunakan untuk mengelola data buku yang meliputi informasi seperti judul, penulis, penerbit, tahun, jumlah, dan rating. Pertama, data buku dimasukkan oleh pengguna. Setelah itu, program menampilkan buku dengan rating tertinggi. Selanjutnya, seluruh data diurutkan berdasarkan rating dari yang terbesar ke yang terkecil menggunakan metode insertion sort. Dari hasil pengurutan tersebut, ditampilkan maksimal lima buku dengan rating tertinggi. Terakhir, program melakukan pencarian buku berdasarkan rating tertentu

menggunakan metode pencarian biner. Jika ditemukan, data buku akan ditampilkan, dan jika tidak, akan diberikan pesan bahwa data tidak ditemukan.

D. Kesimpulan

Pada praktikum ini, telah dibuat program untuk mengelola data buku yang mencakup proses input, pengolahan, hingga pencarian data. Data buku yang dimasukkan kemudian digunakan untuk menentukan buku dengan rating tertinggi, mengurutkan seluruh data berdasarkan rating secara menurun menggunakan metode insertion sort, serta menampilkan beberapa buku dengan nilai rating terbaik. Selain itu, program juga dilengkapi dengan proses pencarian data menggunakan metode binary search untuk menemukan buku berdasarkan rating tertentu.

Melalui praktikum ini dapat disimpulkan bahwa penggunaan algoritma pengurutan dan pencarian sangat membantu dalam mengelola data secara lebih terstruktur dan efisien. Insertion sort digunakan untuk menyusun data agar memudahkan proses selanjutnya, sedangkan binary search memungkinkan pencarian dilakukan dengan lebih cepat pada data yang sudah terurut. Dengan demikian, kombinasi kedua metode ini dapat meningkatkan kinerja program dalam pengolahan data.